NLP CODING EXERCISES

Final Report

Student: Jesse Steven

Date: June 30, 2026

A comprehensive review of Natural Language Processing exercises completed during coursework, including text preprocessing, named entity recognition, sentiment analysis, and machine translation demonstrations.

Table of Contents

Exercise 1: Text Pre-processing

Github/Hugging Face link: https://github.com/ZarcDmC01/NLP/blob/main/Exercie_1.ipynb

Demo link: Interactive Jupyter Notebook with pyLDAvis visualization

What I learned:

During this exercise, I learned comprehensive text preprocessing techniques essential for NLP tasks:

• **Case Normalization:** Converting text to lowercase to ensure consistency across the dataset
• **Tokenization:** Breaking text into individual tokens using gensim's simple_preprocess function
• **Stopwords Removal:** Filtering out common words (the, is, at, etc.) to focus on meaningful content
• **Stemming & Lemmatization:** Reducing words to their root forms using SnowballStemmer and WordNetLemmatizer
• **Topic Modeling (LDA):** Implementing Latent Dirichlet Allocation to discover hidden topics in documents
• **Vocabulary Management:** Creating dictionaries, filtering extremes, and managing corpus representations
• **Bag-of-Words (BoW):** Converting documents into numerical representations for machine learning
• **Model Evaluation:** Using coherence scores (C_V metric) to assess topic quality and model performance

Challenges:

• **Balancing Preprocessing:** Finding the right balance between aggressive and gentle preprocessing to preserve meaning while removing noise
• **Empty Documents:** Handling documents that become empty after aggressive preprocessing (73 documents removed)
• **Hyperparameter Tuning:** Optimizing LDA parameters (num_topics, passes, iterations) for best coherence score
• **Vocabulary Size:** Managing large vocabularies - reduced from initial size to 1000 most relevant terms
• **Coherence Interpretation:** Understanding that coherence scores alone don't guarantee meaningful topics

What needs to be improved:

• **Interactive Visualization:** Implement more advanced topic visualization beyond pyLDAvis
• **Domain-Specific Stopwords:** Create custom stopword lists tailored to specific domains for better topic extraction
• **Optimal Topic Count:** Implement automated methods to determine the optimal number of topics instead of manual selection
• **Performance Optimization:** Optimize for larger datasets using more efficient algorithms
• **Cross-validation:** Implement k-fold cross-validation to ensure preprocessing robustness

Exercise 2: Custom NER using spaCy

Github/Hugging Face link: https://github.com/ZarcDmC01/NLP/blob/main/NER.ipynb

Demo link: *To be developed*

What I learned:

While the dedicated NER exercise is under development, I explored Named Entity Recognition concepts through topic modeling and text classification:

• **spaCy Framework:** Understanding the power of spaCy for production-grade NLP tasks
• **Entity Recognition:** Learning how to identify specific entities (food, quantities, processes) in unstructured text
• **Pattern Recognition:** Developing techniques to recognize domain-specific patterns in text data
• **Label Management:** Creating and managing custom entity labels for specific use cases
• **Model Training:** Understanding the process of training custom NER models on annotated datasets
• **Rule-Based NER:** Implementing rule-based entity extraction using regular expressions and custom logic

Challenges:

• **Annotation Effort:** Creating properly annotated datasets for training custom models requires significant manual effort
• **Domain Adaptation:** Adapting pre-trained NER models to specific domains while maintaining accuracy
• **Ambiguity Resolution:** Handling ambiguous entities that could belong to multiple categories
• **Entity Boundaries:** Correctly identifying where entities begin and end in text

What needs to be improved:

• **Complete Implementation:** Develop full spaCy custom NER model with proper training pipeline
• **Dataset Creation:** Build annotated corpus for domain-specific entity recognition
• **Multi-Language Support:** Extend NER to handle multiple languages
• **Evaluation Metrics:** Implement precision, recall, and F1-score evaluation framework
• **Integration:** Create inference pipeline for real-time entity extraction

Exercise 3: Sentiment Analysis using TextBlob, VADER, and Flair

Github/Hugging Face link: https://github.com/ZarcDmC01/NLP/blob/main/flair.ipynb | Gradio App

Demo link: Interactive Gradio application comparing 4 sentiment models

What I learned:

This comprehensive sentiment analysis project taught me multiple approaches to determining text sentiment:

• **Lexicon-Based Analysis (TextBlob):** Using pre-built word sentiment dictionaries for fast, interpretable predictions
• **Statistical Approaches (Naive Bayes):** Training models on labeled movie review datasets for context-aware analysis
• **Rule-Based Systems (VADER):** Implementing sophisticated rules handling punctuation, intensity, and negations
• **Deep Learning (Flair):** Leveraging transformer-based neural networks for state-of-the-art accuracy (94%)
• **Dataset Preparation:** Working with real-world sentiment-labeled sentences from UCI ML Repository (748 movie reviews)
• **Performance Metrics:** Evaluating models using accuracy, precision, recall, and F1-score
• **Model Comparison:** Quantitative analysis showing accuracy trade-offs between speed and precision

Challenges:

• **Model Trade-offs:** Balancing speed (TextBlob: fast) vs. accuracy (Flair: 94% vs 75% for VADER)
• **Sarcasm Detection:** All models struggle with sarcastic and ironic statements
• **Context Handling:** Lexicon-based models miss nuanced contextual sentiment shifts
• **Computation Cost:** Deep learning models (Flair) require more computational resources
• **Domain Adaptation:** Movie review models may not generalize well to other domains (Twitter, product reviews)

Results Summary:

Model	Accuracy	Precision	Recall	F1-Score
TextBlob (Pattern)	74.9%	78.1%	74.3%	73.8%
VADER	77.5%	79.9%	77.1%	76.9%
Flair (Deep Learning)	**94.0%**	**94.0%**	**94.0%**	**94.0%**

What needs to be improved:

• **Advanced Techniques:** Implement BERT-based sentiment analysis for even better accuracy
• **Aspect-Based Sentiment:** Extract sentiment towards specific aspects in reviews (e.g., "good plot, bad acting")
• **Ensemble Methods:** Combine multiple models to improve robustness
• **Real-time Processing:** Optimize for streaming data and real-time sentiment monitoring

- **Explainability:** Add SHAP or LIME for model interpretability

Exercise 4: Building a Machine Translation Demo with LLMs and Gradio

Github/Hugging Face link: https://github.com/ZarcDmC01/NLP/blob/main/paraphraser_demo.ipynb

Demo link: Interactive Gradio web interface for real-time translation

What I learned:

This exercise demonstrated building production-ready NLP applications with user-friendly interfaces:

• **Gradio Framework:** Creating interactive web interfaces for NLP tasks without web development knowledge
• **LLM Integration:** Leveraging pre-trained language models for translation and paraphrasing tasks
• **Text Generation:** Understanding how transformers generate human-quality translations
• **Evaluation Metrics:** Using BLEU and ROUGE scores to quantify translation quality
• **BLEU Score:** Measuring n-gram precision to compare translations against references
• **ROUGE Score:** Evaluating recall-based metrics for translation quality assessment
• **UI/UX Design:** Creating intuitive interfaces that make NLP tools accessible to non-technical users
• **Deployment:** Understanding how to deploy models as shareable applications

Challenges:

• **Model Size:** Large translation models require significant GPU memory for real-time inference
• **Latency:** Balancing model accuracy with response time for interactive applications
• **Multi-language Support:** Handling diverse language pairs and morphological variations
• **Metric Interpretation:** BLEU scores don't always correlate with human perceived quality
• **Domain Adaptation:** Generic models may produce poor translations for specialized domains

What needs to be improved:

• **Language Pairs:** Expand to support 50+ language combinations beyond initial implementation
• **Fine-tuning:** Fine-tune models on domain-specific corpora for specialized translations
• **Error Handling:** Implement robust error handling and fallback mechanisms
• **Batch Processing:** Enable efficient processing of large document collections
• **Quality Filtering:** Add confidence scores and filtering for low-quality translations
• **User Feedback:** Implement feedback mechanisms to continuously improve model performance

Conclusion and Recommendations

This comprehensive set of NLP exercises provided hands-on experience with core techniques and modern approaches in natural language processing. The progression from fundamental text preprocessing through advanced deep learning models demonstrates the evolution and sophistication of NLP applications.

Key Takeaways:
1. **Foundation Matters:** Proper text preprocessing significantly impacts downstream task performance
2. **Model Selection:** Different approaches (lexicon-based, statistical, neural) have different trade-offs
3. **Evaluation is Critical:** Multiple metrics are necessary to fully understand model behavior
4. **User Experience:** Tools like Gradio democratize access to complex NLP models
5. **Continuous Improvement:** All models have limitations and benefit from domain adaptation and fine-tuning

Future Learning Paths:
• Advanced transformer architectures (BERT, GPT, T5)
• Multi-task learning and transfer learning
• Few-shot and zero-shot learning approaches
• Multimodal NLP combining text with images and audio
• Production deployment and monitoring of NLP systems